Effective Java Notes

Item 18 — Favor Composition Over Inheritance

Main Idea

Prefer **Composition** over **Inheritance** for code reuse.

Why Inheritance is Dangerous?

- Inheritance breaks **Encapsulation**.
- A subclass depends on the superclass's internal implementation.
- Changes in the superclass may break subclasses.
- Leads to **Fragile Base Class Problem**.

Key Rule

Use inheritance only when:

```
B IS-A A
```

Example:

```
Dog IS-A Animal ✅
```

Avoid inheritance when:

```
B HAS-A A
```

Example:

```
Car HAS-A Engine ✅
```

Use Composition:

```
class Car {
    private Engine engine;
}
```

Composition

Instead of:

```
class B extends A
```

Use:

```
class B {
    private A a;
}
```

Benefits of Composition

- Better encapsulation.
- More flexible.
- Less fragile.
- Easier maintenance.
- Independent of superclass implementation details.

Forwarding

Forward calls to the wrapped object:

```
public void add(E e) {
    set.add(e);
}
```

Decorator Pattern

Add functionality by wrapping an object instead of inheriting from it.

Takeaway

> Favor Composition Over Inheritance.

Item 19 — Design and Document for Inheritance or Else Prohibit It

Main Idea

If a class allows inheritance:

- Design it specifically for inheritance.
- Document it carefully.

Otherwise:

- Prohibit inheritance.

Self-Use

A method calling another overridable method inside the same class.

Example:

```
public void methodA() {
    methodB();
}
```

If `methodB()` is overridden:

```
@Override
public void methodB() {
}
```

Then `methodA()` behavior changes.

Rule

Document all self-use patterns.

Document:

- Which methods call other overridable methods.
- In what order.
- Their effect on behavior.

Constructors Rule

Never call overridable methods from constructors.

Bad:

```
class Parent {

    Parent() {
        print();
    }

    public void print() {}
}
```

Reason:

```
Parent constructor executes first.
Child state is not initialized yet.
```

May cause:

```
null
NullPointerException
Unexpected behavior
```

Protected Members

Use protected members only when necessary.

Too many protected members:

- Expose implementation details.
- Restrict future changes.

Test Inheritance

Before releasing a class:

```
Write subclasses.
Test real inheritance scenarios.
```

If You Don't Need Inheritance

Use:

```
final class MyClass {
}
```

Or:

```
private constructors
```

Key Guidelines

Design for Inheritance

- Document self-use.
- Expose minimal protected members.
- Test with subclasses.
- Never call overridable methods from constructors.

Otherwise

Prevent inheritance.

Exam Notes

Item 18

```
IS-A  → Inheritance
HAS-A → Composition
```

```
Composition > Inheritance (usually)
```

Item 19

```
Design for inheritance
OR
Prohibit inheritance
```

```
Constructors must not invoke overridable methods.
```

```
Document self-use patterns.
```

Golden Quotes

Item 18

> Favor Composition Over Inheritance.

Item 19

> Design and Document for Inheritance or Else Prohibit It.